Tutorial 3 – Local Sampling and Anti-Agents

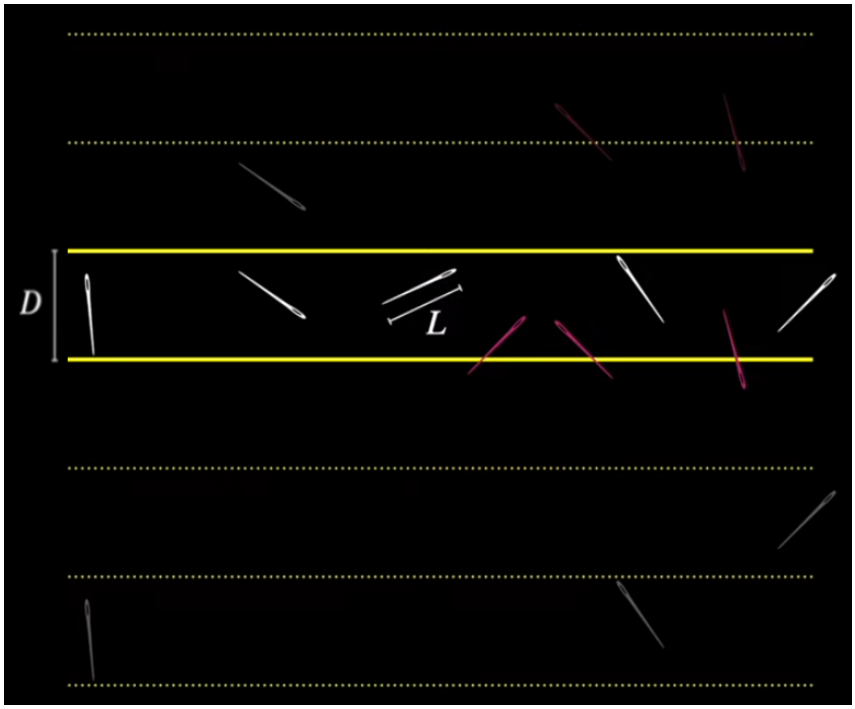
Deadline: 14.06.2026

Objectives:

- Implement a Buffon's needle simulator
- Boost a robot swarm aggregation behavior with so-called 'anti-agents'
- Measure performance in a swarm simulation

1 Buffon's Needle

The Buffon's Needle method estimates π by repeatedly dropping a needle of length L onto a grid of parallel lines spaced D apart and calculating the ratio of how often the needle crosses a line compared to the total number of drops. It is not necessary to simulate the entire setup across an expansive 2D plane. Instead, the problem can be reduced to a single interval of width D, since the center of the needle always lands between two parallel lines. What remains is a simple geometric problem: Determining whether the needle intersects one of the boundary lines based on its angle. To understand this, refer to the video below.



Buffon's Needle Video

a) First, derive the intersection probability P, explain and show how it can be used to estimate π.

b) Implement Buffon's needle experiment for a needle length of $L = 0.7$ and line spacing of $D = 1$. Use your simulation to determine the intersection probability P. Test your program by checking whether it estimates a good approximation to π according to the formula you obtained above.

c) An experiment consists of n needles being thrown. Run experiments with

$$n \in \{10, 20, \ldots, 990, 1000\}$$

trials. For each setting of n, run the experiment 10,000 times and calculate the standard deviation of the resulting set of intersection probabilities. Plot the standard deviation over the number of trials n.

d) For a maximum of $n = 100$ trials, plot the currently measured probability of many experiments over the trial number n along with the Binomial proportion 95%-confidence interval. [1]

e) Measure the ratio of experiments for which the true probability is outside of the 95%-confidence interval based on the measured probability. Plot this ratio over the number of trials n. Interpret your results.

[1] Binomial proportion 95%-confidence interval is defined by:

$$\hat{P} \pm 1.96\sqrt{\frac{1}{n}\hat{P}(1-\hat{P})}.$$

2 Anti-agents in Swarm Aggregation

An interesting approach is that of 'anti-agents' by Scheidler et al. [2] and Merkle et al. [1]. The idea is to have a heterogeneous swarm with a minority of robots that behave basically inversely to the majority of robots. They use it for object clustering and observe an improvement in performance once a certain percentage of anti-agents is added.

The standard approach to object clustering is probabilistic with a probability P_{pick} to pick up an object and P_{drop} to drop it. An anti-agent is a robot that behaves differently than the majority of robots. 'Reverse anti-agents' invert the above probabilities to $1 - P_{\text{pick}}$ and $1 - P_{\text{drop}}$. Scheidler et al. [2] found a counterintuitive effect, namely that certain numbers of reverse anti-agents increase the observed clustering effect.

In this task you can choose from two options, either (a) or (b), but always (c):

a) Implement object clustering (aka 'wood chip clustering,' robots pick up objects and cluster them) and add anti-agents in the above sense. You can also introduce pick-up and drop probabilities that depend on densities of objects (pick-up more likely in sparse areas, drop more likely in dense areas). This would then need to be inverted for the anti-agents as well.

b) Implement a swarm aggregation scenario (robots cluster, no objects) and add anti-agents that we define in the following way. Anti-agents move around and can order individual aggregated robots (via messaging) to leave a cluster. It seems useful to make that 'leave' command dependent on the cluster/neighborhood size.

c) Test different percentages of anti-agents (preferably low percentages) and measure the performance. You can define the performance, for example, as the biggest observed cluster after a defined time. You should do several repetitions for each setup to get statistically relevant measurements.

   Can you confirm the findings of Scheidler et al. [2] and Merkle et al. [1]? It is actually tricky to reproduce their result. Try different parameter sets to find situations when anti-agents really help.

3 Local Sampling in a Swarm

We have a robot swarm of size N with the robots uniformly distributed over the unit square (robot positions (x, y) with $x \in [0, 1]$, $y \in [0, 1]$). Each robot is either black or white with equal probability.

The local sampling is done in the following way. A particular robot perceives the color of all robots (including itself) in the neighborhood. The neighborhood is defined by the sensor range r. A robot belongs to the neighborhood of another robot if their distance is less than the sensor range r.

From the local ratio of black robots the robot estimates the overall number of black robots in the swarm.

Implement this scenario. Do experiments for swarm sizes

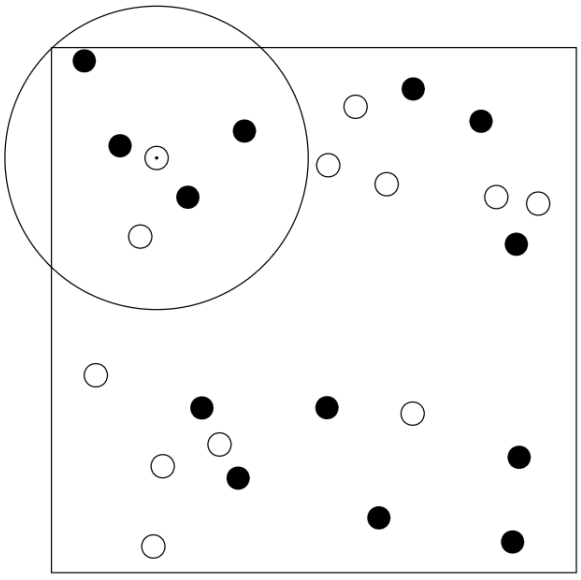
$$N \in [2, 200]$$

each with sensor ranges

$$r \in [0.02, 0.5].$$

For each setting of N and r do 1000 independent experiments and calculate mean and standard deviation of the robot's estimate of black robots in the swarm.

Plot mean and standard deviation for some interesting settings. Try to think of consequences that these measurements would have for an implementation of a robot swarm whose effectivity is directly dependent on these local samplings.



Swarm of black and white robots

References

[1] Daniel Merkle, Martin Middendorf, and Alexander Scheidler. Swarm controlled emergence-designing an anti-clustering ant system. In *2007 IEEE Swarm Intelligence Symposium*, pages 242–249. IEEE, 2007.

[2] Alexander Scheidler, Daniel Merkle, and Martin Middendorf. Swarm controlled emergence for ant clustering. *International Journal of Intelligent Computing and Cybernetics*, 2013.

Submission Requirements:

- Groups can consist of a maximum of 2 students
- Submit one ZIP file named:
  `CollRob_03_YOURLASTNAME1_YOURLASTNAME2.zip`
- Include only necessary files (no large or irrelevant files)
- Ensure the project is **well-structured and clean**
- Include all source code, runnable without complex setup
- Provide a short README explaining how to run the code
- Include plots/screenshots for each implemented behavior
- Include a report (implementation, observations, group members, and completed tasks)
- Include a short video demonstration of the behaviors (.mp4)
- Do not include the original task sheet in the submission